IE540
L7. Value Function Approximation

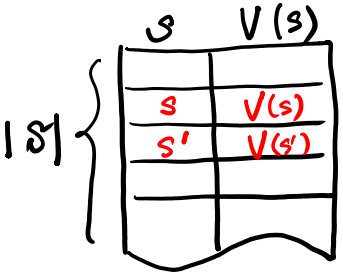
KAIST ISE
Hayong Shin

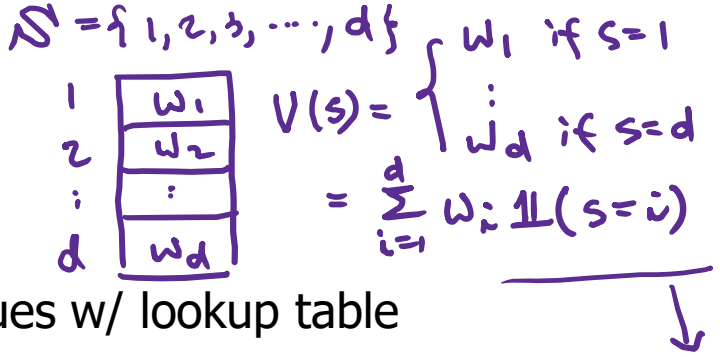
Overview

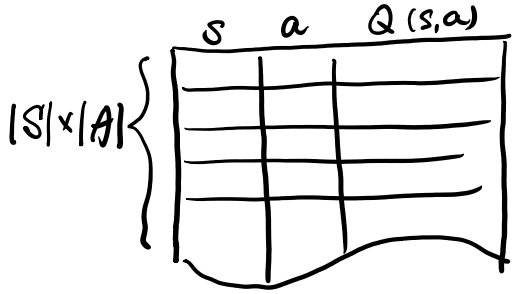
- Model is known → DP (in Lecture 3)
  - Evaluation (prediction) : Iterative policy evaluation
  - Control (finding optimal solution)
    - PI → GPI → VI
    - Async. DP
- Model is unknown → RL (by use of interaction samples)
  - Value based RL
    - MC method : Evaluation, Control (in Lecture 4)
    - TD method : Q, SARSA (in Lecture 5)
    - n-step bootstrapping (in Lecture 6)
    - value function approximation (in Lecture 7)
    - NN, DQN (Lecture 8)
  - Policy based RL (Lecture 9~11)
    - Derivative free methods
    - Policy gradient

Lookup table

- state space
  - dimension $s = (s_1, s_2, \dots, s_d) \rightarrow \dim(s) = d$
  - cardinality $s \in \mathcal{S} = \mathcal{S}_1 \times \mathcal{S}_2 \times \cdots \times \mathcal{S}_d \rightarrow |\mathcal{S}| = |\mathcal{S}_1| \times |\mathcal{S}_2| \times \cdots \times |\mathcal{S}_d|$
- lookup table (= table lookup) function
  - $V(s)$: one entry for each state s, $|\mathcal{S}|$ entries
  - $Q(s,a)$: one entry for each state-action pair, $|\mathcal{S}| \times |\mathcal{A}|$ entries







- 2 important issues w/ lookup table
  - large (or infinite) state space / action space
    - storage space, computation time
    - continuous space/action
  - no generalization
    - learning each state (or state-action) value individually
      - in many case, $V(s) \approx V(s')$ if $s \approx s'$
      - however, each entry is isolated in lookup table

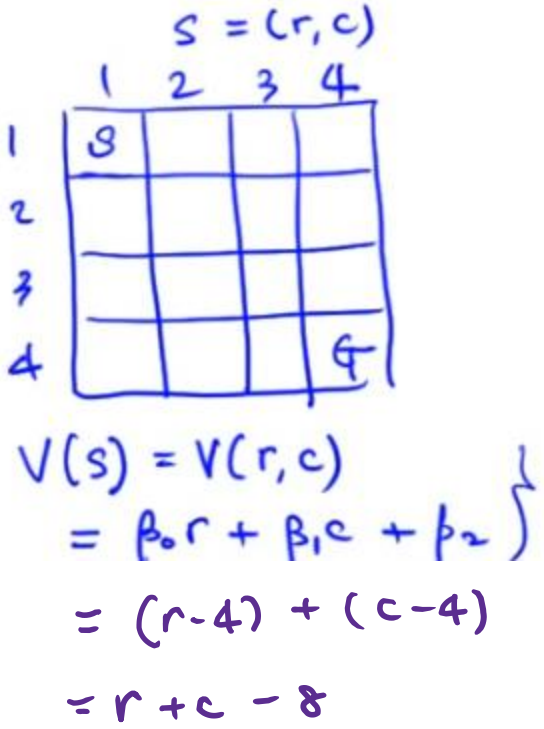
$$\nabla_{w_i} V(s) = \frac{dV(s)}{dw_i} = 1$$

$$\nabla_w V(s) = \underbrace{(1, 1, \cdots, 1)}_{d}$$

Function approximation

- Use function approximation
  - $v(s;w) \approx V(s)$
  - $q(s,a;w) \approx Q(s,a)$
  - w : function parameter
    - $\dim(w) \ll |\mathcal{S}|$
    - find w that best describes $V(s)$ or $Q(s,a)$
      - update w from interaction data

- 2 important decisions on FA
  - features
    - $\mathrm{x}(s)$
  - function form
    - linear combination of features
    - neural net
    - decision tree
    - Fourier, wavelet
    - …



Optimization formulation of FA

- Objective function
  - $J(w) = \frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}}\left(V^{\pi}(s) - v(s;w)\right)^2$, $V^{\pi}(s)$: true vaue function, $v(s;w)$: approx. value
  - $J(w) = \frac{1}{2}\mathbb{E}_{s\sim d^{\pi}(s)}\left[\left(V^{\pi}(s) - v(s;w)\right)^2\right]$, $d^{\pi}(s)$: stationary dist.
- Derivative $\left(\nabla_w J(w) = \left(\frac{\partial J(w)}{\partial w_1}, \frac{\partial J(w)}{\partial w_2}, \dots, \frac{\partial J(w)}{\partial w_d}\right)\right)$

$$\begin{aligned}\nabla_w J(w) &= \frac{1}{2}\nabla_w \mathbb{E}_{d^{\pi}(s)}\left[\left(V^{\pi}(s) - v(s;w)\right)^2\right] \\ &= \frac{1}{2}\mathbb{E}_{d^{\pi}(s)}\left[\nabla_w\left(V^{\pi}(s) - v(s;w)\right)^2\right] \\ &= \mathbb{E}_{d^{\pi}(s)}\left[-\left(V^{\pi}(s) - v(s;w)\right)\nabla_w v(s;w)\right]\end{aligned}$$

TAQ. Under what condition can we interchange $\mathbb{E}$ (or integral) and ∇_w (differentiation)?

  - Sample gradient : $\hat{\nabla}_w J(w) = -\left(V^{\pi}(s) - v(s;w)\right)\nabla_w v(s;w)$
    - TAQ. Do we know $V^{\pi}(s)$?
- Optimization $\min J(w)$: $w \leftarrow w - \alpha\nabla_w J(w)$
  - Gradient descent : $\Delta w = -\alpha\nabla_w J(w) = \alpha\mathbb{E}_{d^{\pi}(s)}\left[\left(V^{\pi}(s) - v(s;w)\right)\nabla_w v(s;w)\right]$
  - S.G.D. : $\Delta w = -\alpha\hat{\nabla}_w J(w) = \alpha\left(V^{\pi}(s) - v(s;w)\right)\nabla_w v(s;w)$

Incremental prediction (policy eval.) w/ VFA

- Stochastic Gradient Descent
  - Update w at S_t : $w_{t+1} \leftarrow w_t + \Delta w$
    - $\Delta w = -\alpha \hat{\nabla}_w J(w_t) = \alpha\big(V^\pi(S_t) - v(S_t; w_t)\big)\nabla_w v(S_t; w_t)$
  - true target $V^\pi(.)$ is not available
    - MC : use sample return G_t
      - $\Delta w = \alpha\big(G_t - v(S_t; w_t)\big)\nabla_w v(S_t; w_t)$
    - TD(0) : use 1-step TD target $R_t + \gamma v(S_{t+1}; w_t)$
      - $\Delta w = \alpha\big(R_t + \gamma v(S_{t+1}; w_t) - v(S_t; w_t)\big)\nabla_w v(S_t; w_t)$
      - Note that $\nabla_w v(S_{t+1}; w_t)$ is not used. Why not?
    - TD(λ) : use λ-return G_t^λ
      - $\Delta w = \alpha\left(G_t^\lambda - v(S_t; w_t)\right)\nabla_w v(S_t; w_t)$

TD(λ) w/ VFA

- Forward view
  - $G_t^\lambda = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}$, where $\delta_t = R_t + \gamma v(S_{t+1}; w) - v(S_t; w)$
  - $\Delta w = \alpha \left(G_t^\lambda - v(S_t; w_t) \right) \nabla_w v(S_t; w_t)$
- Backward view
  - $\delta_t = R_t + \gamma v(S_{t+1}; w) - v(S_t; w)$
  - $\mathbf{e}_t = \gamma\lambda \mathbf{e}_{t-1} + \nabla_w v(S_t; w_t)$
    - $\mathbf{e}_t$: eligibility of parameter (not state)
    - $dim(\mathbf{e}_t) = dim(w_t)$
  - $\Delta w = \alpha \delta_t \mathbf{e}_t$

TAQ. How do we get this?

TAQ. $dim(\mathbf{e}_t)$?

➢ **Algorithm TD(λ) Policy Evaluation w/ VFA (backward)**

- initialize w
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize $\mathbf{e} \leftarrow 0$, initialize S
  - repeat for each step (until S is $terminal$)
    - choose action $A \sim \pi(.|S)$
    - take action A, observe R, S'
    - $\delta \leftarrow R + \gamma v(S'; w) - v(S; w)$ (if S' is $terminal$, $v(S'; w) = 0$)
    - $\mathbf{e} \leftarrow \gamma\lambda \mathbf{e} + \nabla_w v(S; w)$
    - $w \leftarrow w + \alpha\delta\mathbf{e}$

Control w/ VFA

- MC : use sample return G_t
  - $\Delta w = \alpha\big(G_t - q(S_t, A_t; w_t)\big)\nabla_w q(S_t, A_t; w_t)$

- TD(0) : use 1-step TD target
  - SARSA
    - $\Delta w = \alpha\big(R_t + \gamma q(S_{t+1}, A_{t+1}; w_t) - q(S_t, A_t; w_t)\big)\nabla_w q(S_t, A_t; w_t)$
  - Q-learning
    - $\Delta w = \alpha\left(R_t + \gamma \max_a q(S_{t+1}, a; w_t) - q(S_t, A_t; w_t)\right)\nabla_w q(S_t, A_t; w_t)$

- TD(λ) : use λ-return G_t^λ
  - $\Delta w = \alpha\left(G_t^\lambda - q(S_t, A_t; w_t)\right)\nabla_w q(S_t, A_t; w_t)$
  - Backward view
    - $\delta_t = R_t + \gamma q(S_{t+1}, A_{t+1}; w_t) - q(S_t, A_t; w_t)$
    - $\mathbf{e}_t = \gamma\lambda\mathbf{e}_{t-1} + \boxed{\nabla_w q(S_t, A_t; w_t)}$
    - $\Delta w = \alpha\delta_t\mathbf{e}_t$

Incremental control (opt. policy) algorithm

➢ **Algorithm SARSA w/ VFA**

- initialize w for $q(s,a;w)$
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize S
  - choose action $A \sim \epsilon$−greedy($q(S,a;w)$) at S
  - repeat for each step (until S is $terminal$)
    - take action A, observe R, S'
    - choose action $A' \sim \epsilon$−greedy($q(S',a;w)$)
    - $\delta \leftarrow R + \gamma q(S',A';w) - q(S,A;w)$ (if S' is $terminal$, $q(S',A';w) = 0$)
    - $w \leftarrow w + \alpha\delta\nabla_w q(S,A;w)$
    - $S \leftarrow S',\ A \leftarrow A'$

➢ **Algorithm Q-Learning w/ VFA**

- initialize w for $q(s,a;w)$
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize S
  - repeat for each step (until S is $terminal$)
    - Choose action $A \sim \mu(.|S)$: behavior policy, e.g. ϵ−greedy($q(S,a;w)$)
    - take action A, observe R, S'
    - $\delta \leftarrow R + \gamma \max_a q(S',a;w) - q(S,A;w)$ (if S' is $terminal$, $q(S',a;w) = 0$)
    - $w \leftarrow w + \alpha\delta\nabla_w q(S,A;w)$
    - $S \leftarrow S'$

Incremental control (opt. policy) algorithm w/ eligibility trace

➢ **Algorithm SARSA(λ) w/ VFA**

- initialize w for $q(s,a;w)$
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize $\mathbf{e} \leftarrow 0$, initialize S
  - choose action $A \sim \epsilon$−greedy($q(S,a;w)$)
  - repeat for each step (until S is $terminal$)
    - take action A, observe R, S'
    - choose action $A' \sim \epsilon$−greedy($q(S',a;w)$)
    - $\delta \leftarrow R + \gamma q(S',A';w) - q(S,A;w)$ (if S' is $terminal$, $q(S',a;w) = 0$)
    - $\mathbf{e} \leftarrow \gamma\lambda\mathbf{e} + \nabla_w q(S,A;w)$
    - $w \leftarrow w + \alpha\delta\mathbf{e}$
    - $S \leftarrow S',\ A \leftarrow A'$

➢ **Algorithm Q(λ) w/ VFA**

- initialize w for $q(s,a;w)$
- for $k = 1..n$ (n : # of episodes) (or until convergence)
  - initialize $\mathbf{e} \leftarrow 0$, initialize S
  - repeat for each step (until S is $terminal$)
    - choose action $A \sim \mu(.|S)$, take action A, observe R, S'
    - $\delta \leftarrow R + \gamma \max_a q(S',a;w) - q(S,A;w)$(if S' is $terminal$, $q(S',a;w) = 0$)
    - $\mathbf{e} \leftarrow \gamma\lambda\mathbf{e} + \nabla_w q(S,A;w)$
    - $w \leftarrow w + \alpha\delta\mathbf{e}$
    - $S \leftarrow S'$

Features

- Feature vector : replaces state in VFA
  - $s = (s_1, s_2, \dots, s_d) \in \mathcal{S}$: state
  - $\mathrm{x}(s) = \left(\mathrm{x}_1(s), \mathrm{x}_2(s), \dots, \mathrm{x}_n(s)\right)^T$: feature vector
  - eg. chess
    - state : $s = (s_1, s_2, \dots, s_{32}, \mathrm{turn})$
      - s_i : location of piece i $(i = 1$: black king, …)
      - turn = 0 if black's turn
    - feature : $\mathrm{x}(s) = \left(\mathrm{x}_1(s), \mathrm{x}_2(s), \dots, \mathrm{x}_n(s)\right)^T$
      - $\mathrm{x}_1(s)$: # of black knights
      - $\mathrm{x}_2(s)$: # of black bishops
      - $\mathrm{x}_3(s)$: # of black pawns
      - …
  - eg. curling
    - state : $s = (s_1, s_2, \dots, s_{16}, \mathrm{end}, \mathrm{turn})$
      - s_i : location of stone i $(i = 1$: red's 1st stone, …)
      - end : inning #
      - turn = 0 if team red's turn
    - feature : $\mathrm{x}(s) = \left(\mathrm{x}_1(s), \mathrm{x}_2(s), \dots, \mathrm{x}_n(s)\right)^T$
      - $\mathrm{x}_1(s)$: # of red stones in house(circle)
      - $\mathrm{x}_2(s)$: # of red stones remaining to be thrown
      - $\mathrm{x}_3(s)$: red is closest or not
      - …





Feature construction

- Polynomial (monomial) terms
  - eg. $s = (s_1, s_2)$
    $\rightarrow \mathrm{x}(s) = (1, s_1, s_2, s_1^2, s_2^2, s_1 s_2)$: quadratic regression
- other well known functional terms
  - $\mathrm{x}_i(s) = \ln(s)$
  - $\mathrm{x}_i(s) = 1/s$
- Fourier basis : useful for periodic function
  - $\mathrm{x}_i(s) = cos(i\pi s)$

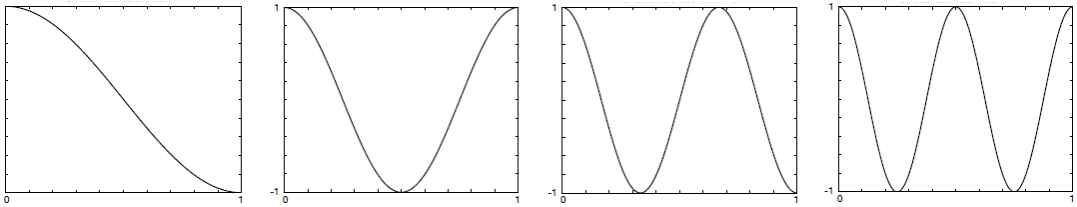
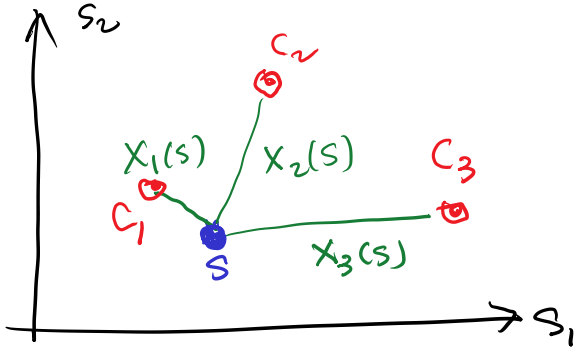


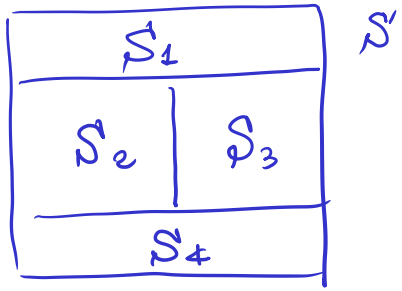
Figure 9.3: One-dimensional Fourier cosine-basis features x_i, $i = 1, 2, 3, 4$, for approximating functions over the interval $[0, 1]$. After Konidaris et al. (2011).

- Radial basis function $\mathrm{x}_i(s) = \exp\left(-\frac{\|s - c_i\|^2}{\sigma_i^2}\right)$
- ...

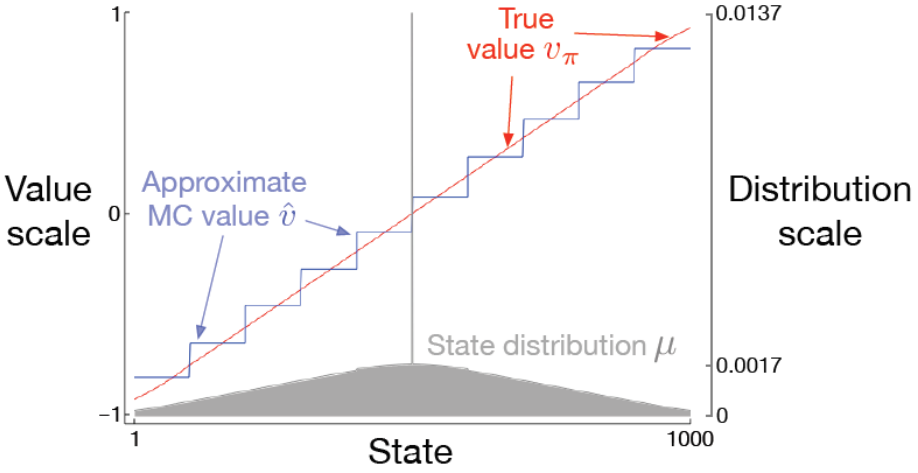


Feature construction

- State aggregation
  - partition $\mathcal{S}$ into disjoint subsets $\mathcal{S} = \mathcal{S}_1 \cup \cdots \cup \mathcal{S}_g$, where $\mathcal{S}_i \cap \mathcal{S}_j = \emptyset$ if $i \neq j$
  - feature $\mathrm{x}_i(s) = \mathbb{I}(s \in \mathcal{S}_i)$



  - eg. 1000-states random walk
    - aggregate 100 states into 1 feature: $\mathcal{S}_1 = \{1, \ldots, 100\}, \mathcal{S}_2 = \{101, \ldots, 200\}, \ldots$



Feature construction

- Coarse coding

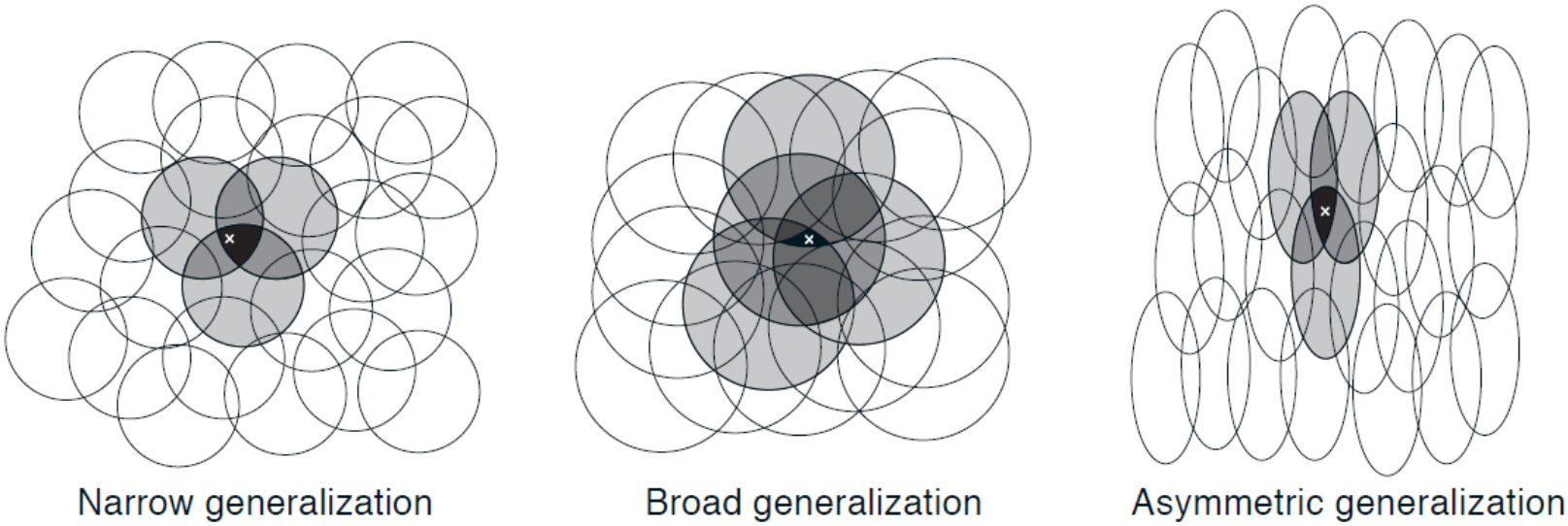
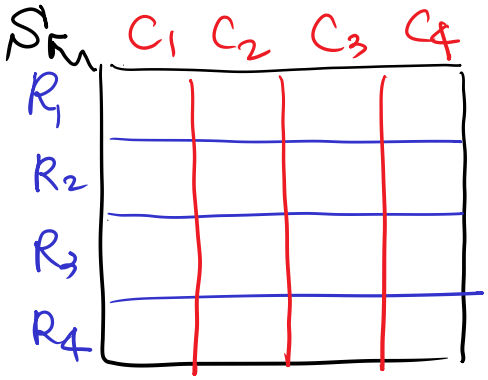


Figure 9.7: Generalization in linear function approximation methods is determined by the sizes and shapes of the features' receptive fields. All three of these cases have roughly the same number and density of features.

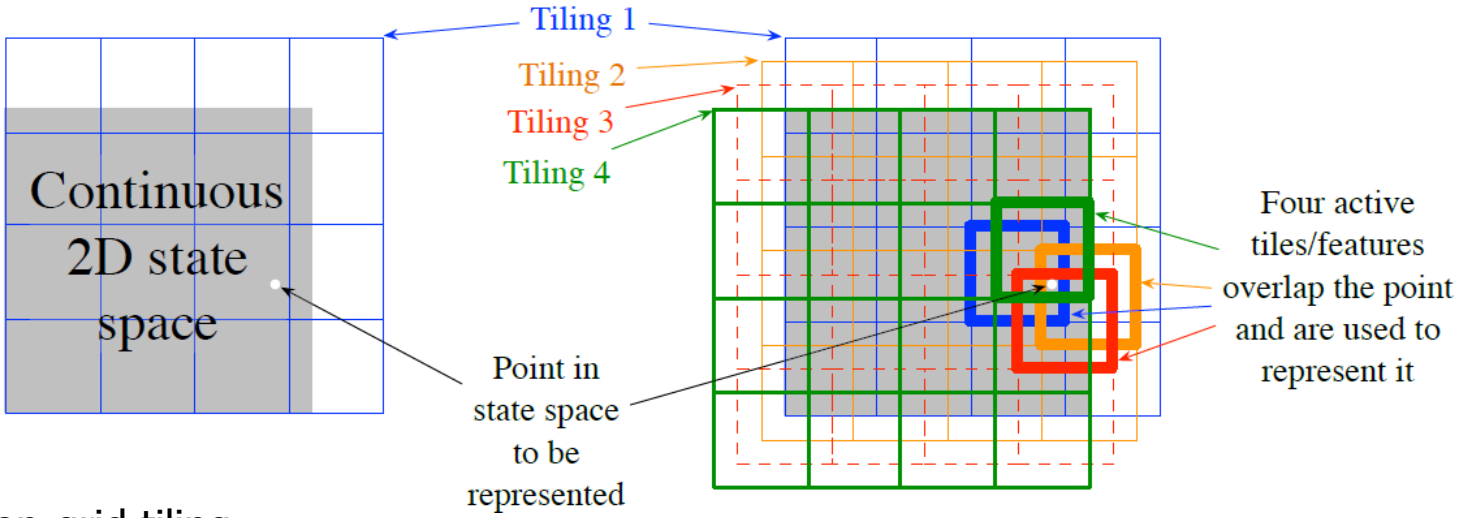
Feature construction

row tiling $S = R_1 \cup R_2 \cup R_3 \cup R_4$

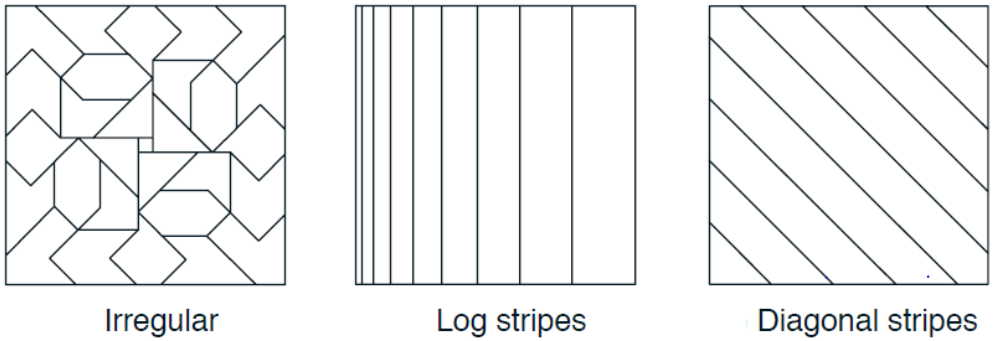
column tiling $S = C_1 \cup C_2 \cup C_3 \cup C_4$



- Tile coding
  - tiling : a partition of state space (= state aggregation)
    - a tile in a tiling : a subset S_i of state space S
  - multiple overlapping tilings



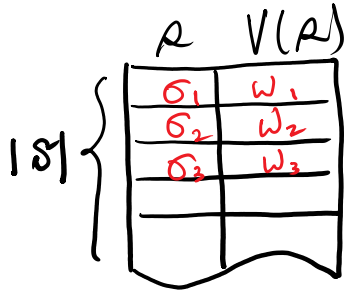
  - non-grid tiling



Linear function approximation

- VFA : feature selection, function form selection
  - well chosen features + appropriate function form
- Linear VFA : very simple function form on (well chosen) features
  - $v(s;w) = w^T \mathrm{x}(s) = \sum_{i=1}^{n} w_i \mathrm{x}_i(s)$
    - $\mathrm{x}(s) = \big(\mathrm{x}_1(s), \mathrm{x}_2(s), \dots, \mathrm{x}_n(s)\big)^T$: state feature vector
    - $\nabla_w v(s;w) = \mathrm{x}(s)$
    - $J(w) = \frac{1}{2}\mathbb{E}_{s\sim d^\pi(s)}\left[\big(V^\pi(s) - v(s;w)\big)^2\right]$
    - $\hat{\nabla}_w J(w) = -\big(V^\pi(s) - w^T\mathrm{x}(s)\big)\mathrm{x}(s)$
    - $\Delta w = -\alpha\hat{\nabla}_w J(w) = \alpha\big(V^\pi(s) - w^T\mathrm{x}(s)\big)\mathrm{x}(s)$
  - $q(s,a;w) = w^T \mathrm{x}(s,a) = \sum_{i=1}^{n} w_i \mathrm{x}_i(s,a)$
    - $\mathrm{x}(s,a) = \big(\mathrm{x}_1(s,a), \mathrm{x}_2(s,a), \dots, \mathrm{x}_n(s,a)\big)^T$: state−action feature vector
    - $\nabla_w q(s,a;w) = \mathrm{x}(s,a)$
    - $J(w) = \frac{1}{2}\mathbb{E}_{s\sim d^\pi(s), a\sim\pi}\left[\big(Q^\pi(s,a) - q(s,a;w)\big)^2\right]$
    - $\hat{\nabla}_w J(w) = -\big(Q^\pi(s,a) - w^T\mathrm{x}(s,a)\big)\mathrm{x}(s,a)$
    - $\Delta w = -\alpha\hat{\nabla}_w J(w) = \alpha\big(Q^\pi(s,a) - w^T\mathrm{x}(s,a)\big)\mathrm{x}(s,a)$

TAQ. Is lookup table a linear VFA?



$S = \{\sigma_1, \sigma_2, \cdots, \sigma_n\}$: State space

σ_i : state value

s : state variable

$\Rightarrow s$ takes a value from S

Indicator function

$\mathbb{1}(\text{cond.}) = \begin{cases} 1 & \text{if cond=true} \\ 0 & \text{otherwise} \end{cases}$

- Yes. Lookup table is a linear VFA.
- What are features used here?

n features $\quad x_i(s) = \mathbb{1}(s = \sigma_i)$

- Write lookup table as a functional form.

$$V(s) = \sum_{i=1}^{n} w_i \, \mathbb{1}(s = \sigma_i) = w^T x(s)$$

- How many parameters are there?

$w = (w_1, \cdots, w_n)$, $w_i = V(\sigma_i)$, $\dim(w) = |S| = n$

- Is update in lookup table equal to update in VFA?

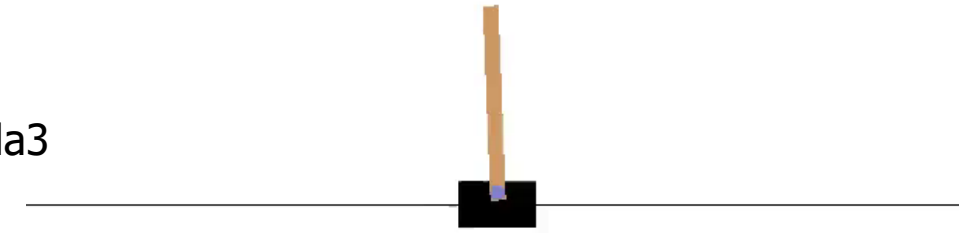
$\Delta w = \alpha\big(V^\pi(s) - v(s;w)\big)\nabla_w v(s;w)$ $\quad v(s;w) = w^T x(s)$ $\quad s = \sigma_i$

$v(s;w) = w_i$ when $s = \sigma_i \Rightarrow \frac{dv}{dw_j} = \begin{cases} 1 & \text{if } i=j \\ 0 & \text{o.w} \end{cases}$ $\nabla_w v(s;w) = [0 \cdots 0 \underset{\downarrow}{1} 0 \cdots 0]^T$

$w \leftarrow w + \Delta w$ changes w_i only $\Rightarrow$ same as lookup table update

Motivating example : OpenAI Gym

- OpenAI Gym
  - toolkit for developing and comparing RL algorithms.
  - many interesting RL task environments are included
  - https://gym.openai.com/



- Installing gym
  - Run Anaconda Prompt for Anaconda3
    - conda install gym

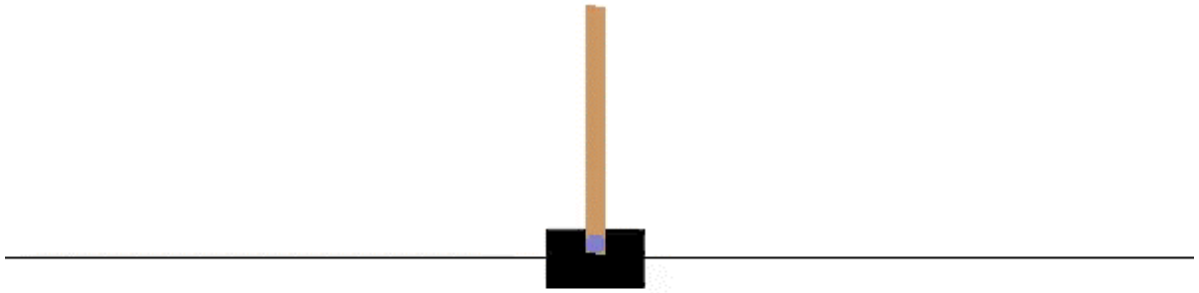
    or
    - pip install gym
  - Check if gym is installed successfully

```
import gym
env = gym.make('CartPole-v1')
env.reset()
for _ in range(1000):
    env.render()
    env.step(env.action_space.sample()) # take a random action
```

Video Recording

- Recording a movie clip
- conda install ffmpeg

```
import gym
env = gym.make('CartPole-v1').unwrapped
env = gym.wrappers.Monitor('CartPole-v1.result', force=True)
env.reset()
for _ in range(1000):
    env.render()
    env.step(env.action_space.sample()) # take a random action
```



Motivating example : Cartole-v1

Observation:

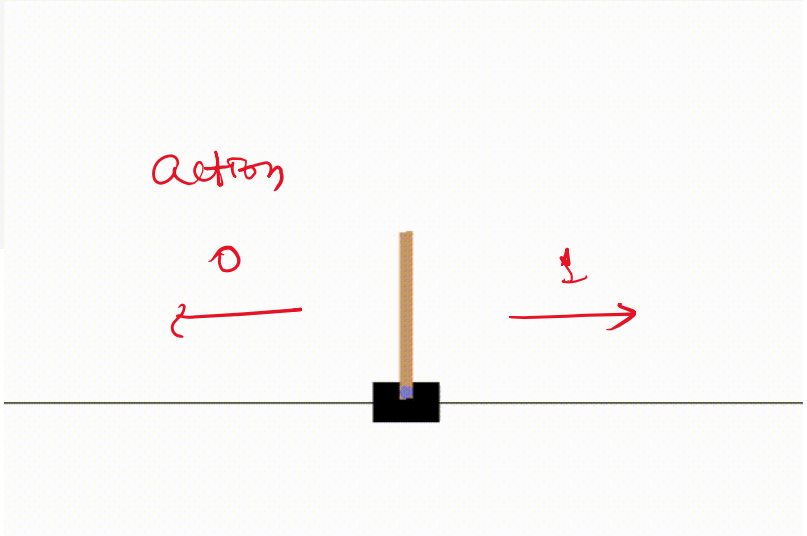
Type: Box(4)

Num	Observation	Min	Max
0	Cart Position	~~-4.8~~ -2.4	~~4.8~~ +2.4
1	Cart Velocity	-Inf	Inf
2	Pole Angle	~~-24~~ deg -12°	~~24~~ deg +12°
3	Pole Velocity At Tip	-Inf	Inf

Actions:

Type: Discrete(2)

Num	Action
0	Push cart to the left
1	Push cart to the right



 https://gym.openai.com/docs/

Modified Cartole-v1 (Cartpole-LinFA.ipynb)

```
Reward:
        Reward is 1 for every step taken, including the termination step

Starting State:
        All observations are assigned a uniform random value in [-0.05..0.05]

Episode Termination:
        Pole Angle is more than 12 degrees.
        Cart Position is more than 2.4 (center of the cart reaches the edge of
        the display).
        Episode length is greater than 200.
        Solved Requirements:
        Considered solved when the average reward is greater than or equal to
        195.0 over 100 consecutive trials.
```

Modified reward : 1+angle*10

```python
def angleReward(obs, r):
    ang = np.abs(obs[2])
    return np.float32(r + 10*ang)
```

Cartpole-v1 : agent w/ simple policy

```python
def simpleAgent(env, policy):
    obs = env.reset()
    i = 1
    done = False
    ret = 0
    while not done:
        env.render()
        action = policy(obs)
        obs, r, done, info = env.step(action)
        r = angleReward(obs, r)
        ret += r
    print (f"Score = {ret}")
```

```python
env = gym.make('CartPole-v1')
simpleAgent(env, policy_random)
simpleAgent(env, policy_angle)
simpleAgent(env, policy_angle_pos)
simpleAgent(env, policy_angle_pos_vel)
env.close()
```

```
Score = 50.39642775058746
Score = 82.24973213672638
Score = 91.42093622684479
Score = 557.9233523607254
```

```python
def policy_random(obs):
    if np.random.random() < 0.5:
        return 0  # left
    else:
        return 1  # right
```

```python
def policy_angle(obs):
    ang = obs[2]
    if ang < 0:
        return 0  # left
    else:
        return 1  # right
```

```python
def policy_angle_pos(obs):
    pos = obs[0]
    ang = obs[2]
    if 10*ang - pos < 0:
        return 0  # left
    else:
        return 1  # right
```

```python
def policy_angle_pos_vel(obs):
    pos = obs[0]
    vel = obs[1]
    ang = obs[2]
    if 10*ang + pos - vel < 0:
        return 0  # left
    else:
        return 1  # right
```

Linear VFA for Cartpole

- State $s = (s_0, s_1, s_2, s_3) = (\text{cart pos.}, \text{cart vel.}, \text{pole angle}, \text{pole tip vel.})$
- Action $a = 0(\text{left}) \text{ or } 1(\text{right})$
- Feature $\mathrm{x}(s, a) = (1, s, (a - 0.5)s)$
- Parameter $w = (w_c, w_s, w_a)$
- $q(s, a; w) = w_c + w_s^T s + (a - 0.5) w_a^T s$

$$q(s, a; w) = \begin{cases} w_c + w_s^T s - 0.5 w_a^T s, & \text{if } a = 0 \\ w_c + w_s^T s + 0.5 w_a^T s, & \text{if } a = 1 \end{cases}$$

- Gradient
  - $\nabla_{w_c} q(s, a; w) = 1$
  - $\nabla_{w_s} q(s, a; w) = s$
  - $\nabla_{w_a} q(s, a; w) = (a - 0.5)s$

- Would it work?
  - check out Cartpole-LinFA.ipynb

Convergence of Control Methods with VFA

- "Deadly Triad" : danger of instability whenever the followings are combined
  - Function approximation : to deal with larger state-action space
    - linear or NN
  - Bootstrapping : update targets that include existing estimates (DP or TD)
    - cf. MC methods rely exclusively on actual returns, not existing estimates
  - Off-policy training : training using behavior policy ≠ target policy

  ➔ all of the above are very important

Algorithm	Tabular	Linear VFA	Nonlinear VFA
MC control	O	(O)	X
SARSA	O	(O)	X
Q-learning	O	X	X

(O) : chatter around near-optimal value function